



UNIVERSIDADE FEDERAL DE SERGIPE

DEPARTAMENTO DE COMPUTAÇÃO

Gustavo Santos Oliveira

Atividade 1

Prof^ª. Glauco de Figueiredo Carneiro

2026

São Cristóvão, Sergipe

Atividade 1 - Sistemas Operacionais

Aluno: Gustavo Santos Oliveira

Trilha de aplicação escolhida: Trilha C: RAG sobre PDF: Ollama + ollama_pdf_rag

Nome completo do modelo: Phi-4-mini-instruct

URL da página do modelo no Hugging Face:

<https://huggingface.co/microsoft/Phi-4-mini-instruct>

<https://huggingface.co/unsloth/Phi-4-mini-instruct-GGUF>

Família, checkpoint/variante e número de parâmetros: Família Phi (Phi-4), variante mini-instruct, 3.8 bilhões de parâmetros.

Formato e quantização utilizados: Formato GGUF, quantização Q4_K_M.

Licença: MIT License.

Parte A - Inventário do Ambiente

Informações do Kernel (uname -a)

Linux server-ollama 5.15.0-191-generic #201-Ubuntu SMP

Versão do Linux (cat /etc/os-release)

PRETTY_NAME="Ubuntu 22.04.5 LTS"

NAME="Ubuntu"

VERSION_ID="22.04"

VERSION="22.04.5 LTS (Jammy Jellyfish)"

Informações do processador (lscpu)

Architecture: x86_64

CPU op-mode(s): 32-bit, 64-bit

CPU(s): 4

Model name: 13th Gen Intel(R) Core(TM) i5-13450HX

Informações da Memória (free -h)

	total	used	free	shared	buff/cache	available
Mem:	7.8Gi	343Mi	6.6Gi	1.0Mi	837Mi	7.2Gi
Swap:	3.9Gi	0B	3.9Gi			

Versão do Ollama (ollama --version)

ollama version is 0.34.0

Ollama List

NAME	ID	SIZE	MODIFIED
nomic-embed-text:latest	0a109f422b47	274 MB	23 hours ago

phi4-mini:latest 78fad5d182a7 2.5 GB 24 hours ago

Ficha da Camada de Aplicação

Nome e URL do repositório	ollama_pdf_rag (https://github.com/tonykipkemboi/ollama_pdf_rag)
Autor	Tony Kipkemboi
Licença	MIT License
Commit, tag ou release	Branch Principal (main)
Data de acesso	12 de setembro de 2026
Finalidade e arquitetura simplificada	É uma aplicação local de RAG (Retrieval-Augmented Generation) focada em leitura e conversação com arquivos PDF mantendo a privacidade dos dados. A arquitetura orquestra o fluxo usando LangChain, divide os documentos em blocos menores (chunking inteligente), converte o texto usando um modelo de embeddings, armazena os índices vetoriais localmente no banco ChromaDB e utiliza o Streamlit como interface gráfica web. A inferência e as respostas são geradas pelo Ollama operando em segundo plano.
Dependências principais	Ollama, Python 3.10+, pip. Bibliotecas: LangChain, Streamlit, ChromaDB, unstructured, PyPDF e numpy.
Comandos de instalação e execução	<pre>git clone https://github.com/tonykipkemboi/ollama_pdf_rag.git python3 -m venv venv source venv/bin/activate pip install -r requirements.txt ollama pull nomic-embed-text python3 run.py</pre>

Observações durante a instalação:

Durante a instalação alguns erros aconteceram:

- Erro de Espaço (LVM): A instalação do requirements.txt inicialmente falhou com [Errno 28] No space left on device devido ao tamanho excessivo das bibliotecas do ecossistema de Inteligência Artificial. Para solucionar, expandi a partição lógica da VM
- Dependências Gráficas Ausentes (libGL.so.1): A biblioteca unstructured acionou um erro crítico ao tentar processar o primeiro arquivo PDF. Isso ocorreu pela ausência de recursos de renderização gráfica do OpenCV, que não vêm instalados por padrão na versão "Server" do Ubuntu. Para solucionar, instalei a dependência.
- Conflito de Versões de Teste: Ocorreu a falha estrutural AttributeError: module 'coverage' has no attribute 'types' devido a incompatibilidades internas entre as bibliotecas do projeto e versões recentes do pacote de testes de código. Para solucionar precisei desinstalar as bibliotecas coverage e pytest para liberar o fluxo de leitura do arquivo.
- Ausência do Diretório de Executáveis no PATH: O comando nativo do web server não foi reconhecido (streamlit: command not found) porque o gerenciador de pacotes o instalou no diretório privado do usuário. Para solucionar, utilizei o caminho absoluto de importação em python, o [python3 run.py](#) e alterei a variável de ambiente global.
- Requisito Oculto de Embeddings: A aplicação encerrou com o status HTTP 404 alegando model "nomic-embed-text" not found no momento de vetorizar o texto. Para solucionar, identifiquei que o o repositório exige um modelo em background exclusivo para embeddings operando além do LLM conversacional. Foi solucionado executando ollama pull nomic-embed-text.
- Durante a execução, a busca vetorial falhou retornando 0 documentos devido à descontinuação do método get_relevant_documents nas versões mais recentes do pacote LangChain. A adaptação realizada foi realizar o downgrade das bibliotecas para a versão 0.1.x, restabelecendo a comunicação entre o banco ChromaDB e o LLM.

Parte B - Processos, Threads e Syscalls

Informações antes de adicionar um PDF e fazer perguntas ao modelo.

Streamlit

PID	NLWP	%CPU	%MEM
-----	------	------	------

1826	10	10.4	10.8
------	----	------	------

Ollama

PID	NLWP	%CPU	%MEM
684	10	0.0	0.4

Rodei o seguinte comando para analisar enquanto eu fazia a pergunta para o modelo:

```
sudo strace -c -p 684 -o strace-resumo.txt
```

Após o modelo dar uma resposta eu parei o strace e rodei o cat strace no resumo.txt e o resultado foi:

```
aluno@server-ollama:~$ cat strace-resumo.txt
```

% time	seconds	usecs/call	calls	errors	syscall
15.52	0.000196	65	3		clock_gettime
13.62	0.000172	172	1		epoll_wait
13.38	0.000169	84	2	1	setsockopt
12.35	0.000156	156	1		sendmsg
10.13	0.000128	128	1		socket
9.58	0.000121	60	2		getsockopt
6.18	0.000078	78	1		close
6.02	0.000076	76	1		getuid
4.43	0.000056	56	1		geteuid
4.43	0.000056	56	1		getegid
4.35	0.000055	55	1		getgid
0.00	0.000000	0	1		read
0.00	0.000000	0	2		gettid
0.00	0.000000	0	1		timerfd_settime
100.00	0.001263	66	19	1	total

1) Família de comunicação de rede e sockets

Syscalls: epoll_wait, socket, setsockopt, getsockopt, sendmsg e read.

O frontend web não executa o modelo diretamente, ele envia a pergunta ao serviço do Ollama através de uma API HTTP local. A chamada socket cria essa conexão de rede interna. O sendmsg é o momento em que o texto foi enviado para o Ollama. O epoll_wait representa o processo aguardando a resposta da IA chegar na placa de rede sem travar as CPU, demonstrando o uso eficiente de entrada/saída não bloqueante pelo sistema operacional.

2) Família de Gerenciamento de Tempo

Syscalls: clock_gettime e timerfd_settime.

A chamada clock_gettime consumiu mais tempo. Isso aconteceu porque as operações de inferência de IA e comunicação de API precisa medir latência constantemente. O sistema operacional é acionado o tempo todo para marcar os timeouts da conexão HTTP e para calcular métricas de desempenho essenciais, como o tempo exato até gerar a resposta.

3) Família de Credenciais e Segurança

Syscalls: getuid, geteuid, getgid e getegid.

Sempre que um processo vai abrir um arquivo, o Linux faz validações de segurança. Essas chamadas verificam a identidade e o grupo do processo para garantir que ele tem as permissões corretas no sistema de arquivos do servidor para acessar os dados do RAG.

Parte C - Experimentos Comparativos

Para realizar essa parte dos experimentos comparativos, utilizei dois pdf's, um pequeno e simples falando sobre a biografia da banda Twenty One Pilots e o segundo um pdf maior com a história da segunda guerra mundial em ordem cronológica.

Dividi em 3 partes para cada um dos pdf's, a indexação para subir o pdf, 2 execuções com uma pergunta simples, 2 execuções com concorrência (abrindo duas páginas), 2 execuções com pergunta de contexto longo.

PDF CURTO:

Tempo Indexar	CPU	RES	MEM	TTFT
15 seg	400	3184	40.1	11s
15 seg	400	3075	38.8	24s
15 seg	100%	Gargalo com mais de 6 min		
15 seg	100%	Gargalo com mais de 6 min		
15 seg	400	3080	38.8	1,35 min

15 seg	400	3083	38.8	1,22 min
--------	-----	------	------	----------

PDF LONGO:

Tempo Indexar	CPU	RES	MEM	TTFT
1,12 min	400	3079	38.8	1,53 min
1,12 min	400	3075	40.3	1,57 min
1,12 min	100%	Gargalo com mais de 6 min		
1,12 min	100%	Gargalo com mais de 6 min		
1,12 min	400	3361	38.8	2,28 min
1,12 min	400	3361	42,3	2,32 min

Questões de Análise

Por que a trilha escolhida é adequada à disciplina?

A trilha C é super importante pois tira a abstração da nuvem e coloca a carga de processamento de inteligência artificial diretamente no hardware local. Praticando os conceitos da disciplina com alocação de memória, escalonamento de processos na cpu e concorrência.

Qual modelo foi selecionado, quais são seus parâmetros e por que foi escolhido?

Foram selecionados dois modelos operando em conjunto: o phi4-mini (LLM para geração de texto, geralmente em torno de 3.8 bilhões de parâmetros) e o nomic-embed-text, que foi exigido durante a execução. Eles foram escolhidos por serem modelos leves, o que permite a execução local via CPU dentro de uma máquina virtual com recursos limitados, mantendo um equilíbrio razoável entre qualidade de resposta e viabilidade de hardware.

Como formato, quantização, contexto e tamanho afetaram armazenamento, RAM ou VRAM?

Como a máquina virtual do Virtual Box não possui GPU, todo o peso caiu sobre a memória RAM e o armazenamento. O carregamento do modelo ocupou uma média base constante de cerca de 3075 MB a 3184 MB de RAM (RES), consumindo entre 38% e 42% da memória do sistema. A indexação saltou de 15 segundos (PDF curto) para 1,12 minutos (PDF longo), mostrando o custo do cálculo vetorial na CPU.

Quais processos e threads foram observados na inicialização e na inferência?

Foram observados o processo do Python (python3 run.py) gerenciando o backend do LangChain/Streamlit, e o processo do Ollama (ollama serve) em background. Durante a geração de respostas, o processo ollama saltou para o topo do htop, abrindo múltiplas threads de cálculo que saturaram o uso da CPU para 400% (indicando a utilização simultânea de 4 núcleos virtuais em carga máxima).

Quais chamadas de sistema (syscalls) foram relevantes para carga, leitura de dados, comunicação ou logs?

As principais syscalls envolvidas na arquitetura foram:

openat, read, write: Manipulação de arquivos de disco durante o upload dos PDFs e a gravação/leitura constante dos índices vetoriais na pasta chroma_db.

mmap: Mapeamento das dezenas de gigabytes dos pesos do modelo para a memória RAM.

socket, connect, sendto, recvfrom: Comunicação de rede interna (via localhost:11434), onde o LangChain (Python) trocava requisições HTTP REST constantemente com a API do Ollama.

O aumento de concorrência melhorou responsividade ou vazão? Quais custos surgiram?

Não melhorou; pelo contrário, inviabilizou o sistema. Conforme demonstrado na tabela de resultados, a Configuração 2 (Concorrência) gerou um gargalo crítico (Timeouts acima de 6 minutos para ambos os PDFs). O custo que surgiu foi o Starvation (inanição) de CPU: sem paralelismo de GPU, o processador não conseguiu lidar com o enfileiramento das múltiplas requisições simultâneas.

Houve competição por CPU, memória, armazenamento, GPU ou rede?

Houve uma contenção severa por CPU e Armazenamento. Devido à ausência de GPU no VirtualBox, todo o cálculo matricial competiu diretamente pelos ciclos da CPU.

Como entradas ou contextos maiores afetaram desempenho e recursos?

A tabela mostra uma degradação clara de desempenho conforme a carga aumentou. O tempo de indexação subiu de 15 segundos (1 página) para mais de 1 minuto (16 páginas). TTFT também subiu entre o pdf curto e o pdf longo. Quando o contexto do pdf longo é pesado, a latência passou a marca de 2,3 minutos.

Como a arquitetura local contribui para privacidade, disponibilidade e controle de dados?

A execução via Ollama em uma VM local garante 100% de privacidade, pois nenhum dado do PDF trafega para servidores de terceiros (como OpenAI ou

Google). A disponibilidade é total, não dependendo de conexão com a internet ou limites de taxa (rate limits) de APIs pagas.

Quais limitações ameaçam a validade dos resultados?

Tive duas limitações, a primeira sendo a impossibilidade de usar a placa de vídeo física para o virtualbox, dificuldade o verdadeiro potencial de velocidade do modelo. E os conflitos de dependências como o método descontinuado de `get_relevant_documents` no langchain e a execução oculta do `multiqueryretriever`, inflando o tempo de processamento.

Quais informações geradas por IA precisaram ser verificadas, corrigidas ou rejeitadas?

Durante os testes antes de limpar o cache, a resposta trouxe ferramentas como o trello que não estavam no escopo do pdf, o que exigiu uma limpeza manual. E as falas recusas, pois a AI afirmou não ter acesso ao documento quando na verdade tinha, mas era um erro do langchain que estava dificultando.

Uso crítico de IA generativa

Ferramenta e modelo utilizados

Para a execução da Trilha C (Experimento Local): Ollama executando os modelos locais `phi4-mini:latest` (LLM) e `nomic-embed-text:latest` (Embeddings) no Ubuntu Server.

Para apoio ao desenvolvimento e documentação: Assistente Virtual Gemini (Modelo Gemini 1.5 Pro).

Finalidade de cada uso

Interpretação e diagnóstico de logs de erro do sistema (Tracebacks do Python/LangChain). Compreensão de métricas avançadas de monitoramento de processos no Linux (interpretação de colunas específicas do comando `htop`). Suporte para recuperação de desastres no sistema operacional (recuperação do boot do Ubuntu via GRUB e `fsck`). Estruturação da metodologia de coleta de dados para garantir o isolamento das variáveis de teste (limpeza de cache do banco vetorial).

Prompts utilizados

"Como eu faço isso do `htop` quando eu for testar, ainda não entendi" (Para aprender a extrair métricas de CPU/RAM em tempo real).

"Apareceu um erro no terminal: 'MultiQueryRetriever' object has no attribute 'get_relevant_documents'. Como eu resolvo?" (Para debugar o erro de versão da biblioteca).

"Testei várias vezes, e precisei fechar e abrir a vm pq tava muito travada. Agora tá travado na begin: loading essential drivers" (Para buscar comandos de recuperação do SO).

"Quais são os campos que eu devo ficar de olho para anotar na tela do htop?" (Para refinar a coleta de dados de threads e RAM real - RES).

"Como formato, quantização, contexto e tamanho afetaram armazenamento, RAM ou VRAM?" (Para auxiliar na fundamentação teórica das métricas coletadas empiricamente).

Sugestões aproveitadas, corrigidas ou rejeitadas

Aproveitadas: A sugestão de acessar o Recovery Mode do Ubuntu para rodar o fsck e corrigir a corrupção do disco virtual salvou a máquina de testes. A edição direta do código-fonte (substituindo `get_relevant_documents` por `invoke`) via editor nano corrigiu o fluxo da aplicação.

Corrigidas/Rejeitadas: A sugestão inicial da IA de realizar o downgrade das bibliotecas LangChain via pip install foi testada, mas rejeitada após os logs confirmarem que não surtiu efeito prático no ambiente, forçando a adoção da intervenção manual no código.

Erros encontrados

A IA local alucinou a presença de ferramentas como Trello e Asana devido à contaminação de cache vetorial antigo. Também houve falsas recusas de resposta geradas por falhas na integração de versão do LangChain, além do travamento completo do processo (Timeout/Starvation) ao testar a concorrência na CPU.

Testes, documentação ou observações usados para verificar respostas

Nenhuma resposta técnica fornecida pela IA de apoio foi assumida como verdadeira sem teste prático. As sugestões de diagnóstico foram cruzadas com a leitura atenta dos logs gerados no terminal da aplicação. As explicações sobre consumo de recursos foram atestadas visualmente pela ferramenta de monitoramento **htop** em tempo real durante a execução da carga. A verificação de integridade do banco de dados foi validada por meio de comandos no shell (`rm -rf chroma_db` seguido de reindexação).

